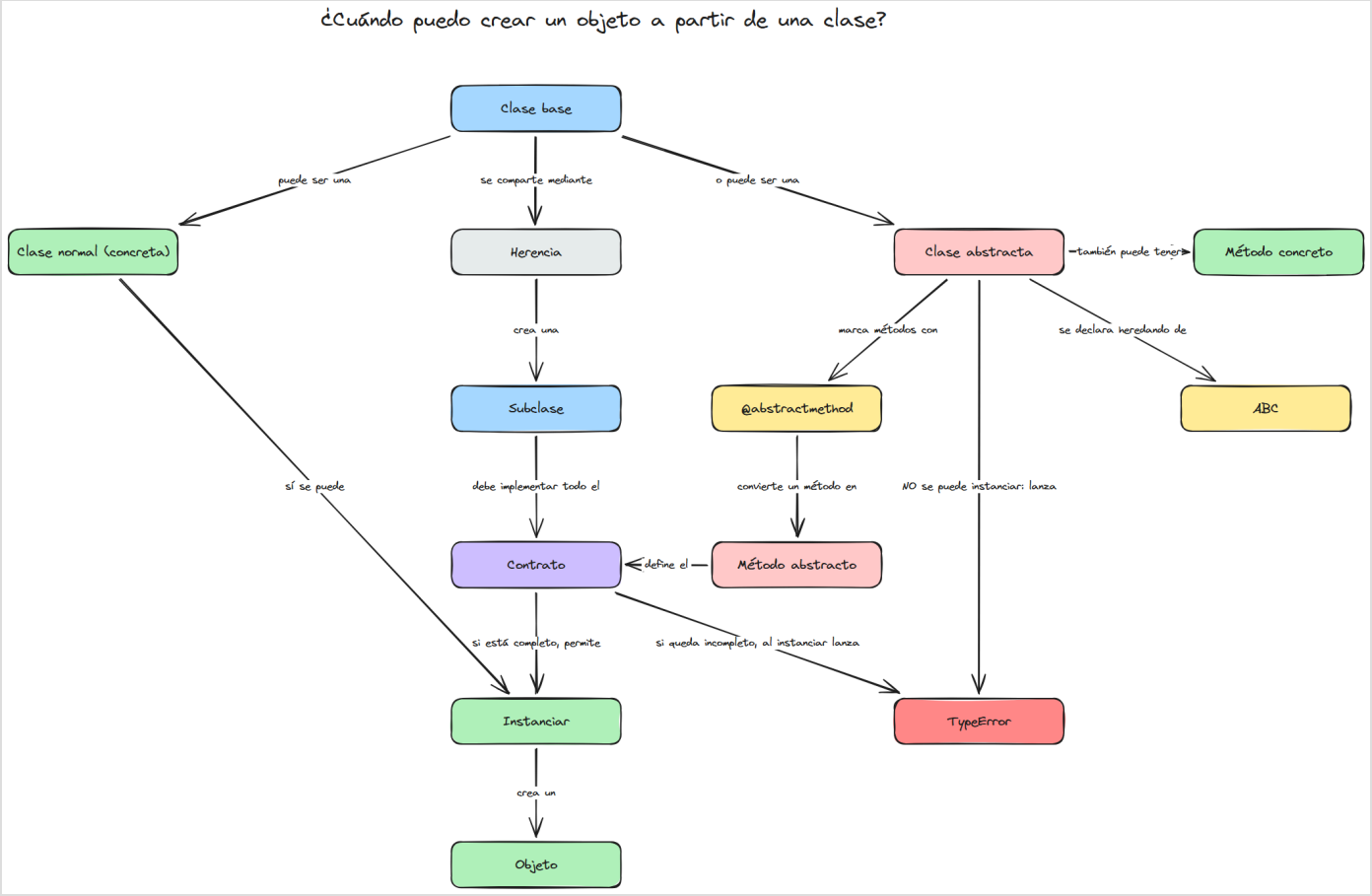
Clases abstractas

Práctica · POO · 14 de octubre de 2026

Qué me pidieron resolver

Leer la guía de clases abstractas, hacer un mapa conceptual en Excalidraw que responda *¿cuándo puedo crear un objeto a partir de una clase?*, resolver los ejercicios interactivos (predecir antes de ejecutar) y registrar lo aprendido en esta bitácora.

Mapa conceptual



Autoevaluación (sección 5)

1. **¿Qué dos cosas necesito para que una clase sea abstracta?** Que herede de `ABC` y que tenga al menos un método marcado con `@abstractmethod`.
2. **Si una subclase no implementa el método abstracto, ¿puedo crear objetos de ella?** No. Sigue siendo abstracta porque el contrato quedó incompleto, y Python lanza `TypeError` al instanciarla.
3. **¿Una clase abstracta puede tener métodos con código normal?** Sí, puede tener métodos concretos que las subclases heredan tal cual.
4. **¿Cuándo aparece el error?** Al instanciar, no al definir la clase.

Resultado de los ejercicios

Lo que predije vs lo que pasó

Ejercicio	Mi predicción	Resultado real	¿Acerté?
1	Falla solo AnimalAbs()	Falla solo AnimalAbs()	✅
2	TypeError: Cuadrado sigue siendo abstracta porque falta perimetro()	TypeError: Cuadrado sigue siendo abstracta porque falta perimetro()	✅
3A	Corre sin error	Corre sin error	✅
3B	TypeError: faltan argumentos del constructor	TypeError: faltan argumentos del constructor	✅
3C	TypeError: la clase sigue siendo abstracta	TypeError: la clase sigue siendo abstracta	✅

Resultado: 5 de 5. El que más me hizo dudar fue el **3C**: `calcular_area()` parece cumplir, pero el nombre no coincide con `area()`, así que el contrato sigue incompleto.

Mi error principal

Pensaba que un método vacío con `pass` ya volvía abstracta a una clase, y que el `TypeError` salía hasta que llamaba al método que faltaba. En realidad lo que la vuelve abstracta es `ABC` + `@abstractmethod`, y Python revisa el contrato **al instanciar**. Tampoco tenía claro que el nombre tiene que ser idéntico: `calcular_area()` no cuenta como `area()`.

La regla con mis palabras

Una clase abstracta es un molde que dice *qué* métodos deben existir pero no *cómo* funcionan, así que no puedo crear objetos de ella. Solo puedo crear objetos de una subclase que escriba **todos** los métodos abstractos con el mismo nombre. Si falta uno, Python me avisa con un `TypeError` en el momento de instanciar.

Mi propio ejemplo

```
from abc import ABC, abstractmethod

class Notificacion(ABC):
    def __init__(self, destinatario):
        self.destinatario = destinatario

    @abstractmethod
    def enviar(self, mensaje):
        pass

    def resumen(self):  # método concreto: lo heredan todas
```

```
        return f"Notificación para {self.destinatario}"

class Correo(Notificacion):
    def enviar(self, mensaje):
        print(f"Enviando correo a {self.destinatario}: {mensaje}")

class SMS(Notificacion):
    pass  # olvidé implementar enviar()

c = Correo("ana@mail.com")
c.enviar("Tu pedido llegó")   # funciona
print(c.resumen())            # Notificación para ana@mail.com

# Notificacion("x")  -> TypeError: clase abstracta
# SMS("3312345678") -> TypeError: falta implementar 'enviar'
```

Conceptos que usé

- Clase abstracta
- Herencia básica
- Polimorfismo
- Excepciones

Una pregunta que me queda

Si una clase hereda de `ABC` pero no tiene ningún método con `@abstractmethod`, ¿sigue siendo abstracta o sí se puede instanciar?